Progetto 1: 3D Perception

Alessandro La Mantia

1. Titolo e Panoramica del Progetto

3D Perception con PointNet: Dalla Classificazione ModelNet40 al Fine-Tuning sui Cavalli

- **Ambiente e Strumenti:** VS Code, Python, PyTorch, Streamlit, Plotly.
- **Obiettivi Chiave:**
  - Comprendere ed elaborare dati 3D non strutturati (Point Clouds) tramite l'architettura della rete neurale PointNet.
  - Sviluppare un'interfaccia interattiva per l'esplorazione e l'ispezione dei meccanismi interni della rete.
  - Condurre un esperimento di fine-tuning su una nuova classe non vista (`horse`) e analizzare in modo critico il trade-off con le classi preesistenti.

2. Il Processo di Apprendimento (Learning Process)

Metodologia di Lavoro e Approccio Incrementale

- **Fase 1 — Studio Teorico:** Comprensione delle sfide della geometria 3D (invarianza alla permutazione, invarianza alla rotazione) e dell'architettura PointNet.
- **Fase 2 — Pipeline di Inferenza e Ispezione:** Costruzione della lettura dati Parquet, preprocessing e visualizzazione delle feature interne.
- **Fase 3 — Sviluppo App Interattive:** Creazione della dashboard Streamlit/Plotly per rendere visibili i concetti teorici.
- **Fase 4 — Sperimentazione Critica:** Progettazione dell'esperimento di adattamento a una nuova classe (cavalli) e valutazione quantitativa del compromesso di accuratezza.

3. I Dati 3D: Nuvole di Punti e Dataset ModelNet40

Rappresentazione Geometrica Diretta

- **Point Cloud (Nuvola di Punti):** Insieme non ordinato di punti 3D definiti dalle coordinate `(X, Y, Z)`. Conserva la geometria continua senza la complessità di griglie 3D (Voxel) o proiezioni 2D.
- **Dataset ModelNet40:** Nel dataset ModelNet40 ci sono 12.308 oggetti CAD, suddivisi in 40 categorie:
  - 9.840 oggetti di training.
  - 2.468 oggetti di test.
- **Sfida Principale:** L'ordine dei punti nel file è arbitrario; la rete deve produrre lo stesso risultato indipendentemente da come i punti vengono ordinati.

4. Preprocessing Geometrico Essenziale

Campionamento, Centratura e Normalizzazione

- **Campionamento Uniforme:** Riduzione a 1.024 punti per campione per garantire uniformità di input ed efficienza computazionale.
- **Normalizzazione Spaziale:** Traslazione del centro della figura nell'origine e scalatura entro una sfera di raggio unitario.

```
def prepara_punti(points, num_punti=1024, seed=42):
    # Generatore casuale con seed fisso: sempre lo stesso campionamento
    rng = np.random.default_rng(seed)

    # Seleziona casualmente num_punti punti dalla point cloud senza ripetizioni
    indices = rng.choice(len(points), num_punti, replace=False)
    selected = points[indices]

    # Sposta la point cloud in modo che il suo centro coincida con l'origine
    selected = selected - selected.mean(axis=0)

    # Calcola la distanza massima dei punti dall'origine
    radius = np.linalg.norm(selected, axis=1).max()

    # Normalizza
    return selected / radius if radius > 0 else selected
```

5. Architettura PointNet: Concetti Chiave

Come PointNet Gestisce l'Ordine e la Trasformazione dei Punti

- **1. Moduli T-Net (Spatial Transformers)**
  - **Trasformazione appresa:** calcola matrici **3×3** e **64×64** per trasformare coordinate e caratteristiche.
  - **Adattamento:** può **ruotare, allungare o comprimere** la point cloud in modo diverso per ogni oggetto.
  - **Obiettivo:** ottenere una rappresentazione più efficace per la **classificazione**.
- **2. Estrazione delle caratteristiche**
  - **Elaborazione punto per punto:** PointNet applica le stesse operazioni a ogni punto in modo **indipendente**.
  - **Feature apprese:** ogni punto viene rappresentato da **1.024 feature numeriche**.
- **3. Global Max Pooling**
  - Per ogni feature viene selezionato il **valore massimo tra tutti i 1.024 punti**.
  - Il risultato è un unico **vettore globale di 1.024 valori** che rappresenta l'intero oggetto.

6. Architettura PointNet: Struttura Essenziale

Dal Punto Singolo al Descrittore Globale

```
# Struttura concettuale del passaggio Forward in PointNet
def forward(self, x):
    # x ha forma: (Batch, 3, N_punti)
    trans_3x3 = self.input_transform(x)          # T-Net spaziale (3x3)
    x = torch.bmm(trans_3x3, x)                  # Allineamento geometrico
    x = F.relu(self.conv_features(x))             # Estrazione feature locali (64)

    trans_64x64 = self.feature_transform(x)      # T-Net feature (64x64)
    x = torch.bmm(trans_64x64, x)
    x = F.relu(self.conv_high_dim(x))            # Mappatura ad alta dimensione (1024)

    global_descriptor = torch.max(x, dim=2)[0]   # GLOBAL MAX POOLING (Invarianza permutazione)
    logits = self.classifier_mlp(global_descriptor) # Classificazione finale (40 classi)
    return logits
```

7. Ispezione Interna con PyTorch Forward Hooks

Dal Max Pooling alla Classificazione

- **1. Quali punti contribuiscono al descrittore?**
  I *Forward Hooks* permettono di intercettare le attivazioni dei **1.024 canali** prima del Global Max Pooling. Per ogni canale possiamo individuare il punto che ha prodotto il **valore massimo**.
- **2. Cosa indicano i colori?**
  Il colore rappresenta **quanti canali** hanno selezionato quel punto come massimo. Un valore elevato indica che quel punto contribuisce al massimo di più caratteristiche, ma **non rappresenta una percentuale di importanza** e non identifica automaticamente una parte specifica dell'oggetto.
- **3. Dal descrittore alla classificazione**
  Il Global Max Pooling produce un **descrittore globale di 1.024 valori**. I livelli finali elaborano questo vettore e producono **40 punteggi**, uno per ciascuna classe. La classe con il punteggio più alto determina la previsione finale.

8. Applicazione Interattiva Principale

Esplorazione e Visualizzazione 3D (Streamlit & Plotly)

- **Interfaccia Utente (`localhost:8501`):**
  - Rendering 3D manipolabile e ruotabile in tempo reale delle nuvole di punti.
  - Visualizzazione delle probabilità di classificazione (Softmax) per le Top-5 categorie.
  - Ispezione interattiva delle attivazioni del Max Pooling con mappatura di colore sovrapposta al modello 3D.

9. Difficoltà e Soluzioni

Le Principali Sfide Tecniche del Progetto

💾 Gestione dei dati

Dataset 3D molto grande → lettura dei dati **solo quando servono**, evitando di caricare tutto in RAM.

🎲 Campionamento casuale

Campioni diversi potevano modificare la predizione → utilizzo di un **seed fisso** per ottenere risultati riproducibili.

🔄 Invarianza all'ordine dei punti

Verifica che riordinare i punti non cambi la predizione → **test automatici** sul modello.

Fine Tuning

11. Estensione del Dominio: La Classe "Horse"

Motivazione e Domanda di Ricerca

- **Il Contesto:** Il dataset ModelNet40 originale non include la categoria "cavallo" (`horse`).
- **Domanda di Ricerca Strategica:**

  "Quanti campioni della nuova classe sono necessari per insegnarla al modello, e qual è il costo in termini di degrado delle prestazioni sulle 40 classi originali?"
- **Approccio Concettuale:** Valutare la capacità di **Continual Learning** e prevenzione dell'**Oblio Catastrofico** (Catastrophic Forgetting).

12. Preparazione del Dataset e Replay Strategy

Estrazione da Objaverse e Prevenzione dell'Oblio

- **Sorgente Nuova Classe:** Modelli 3D di cavalli estratti da **Objaverse** (dataset LVIS) e campionati a 2.048 punti.
- **Deduplicazione Rigorosa:** Controllo e rimozione dei duplicati tramite hash geometrico delle mesh.
- **Tecnica di Replay (Memory Buffer):**
  - Per evitare che la rete "dimentichi" le 40 classi originali, sono stati inclusi nel training set di fine-tuning **5 campioni per ciascuna vecchia classe** (200 campioni totali di replay).
- **Test Set Fisso di Valutazione:** 420 oggetti trasversali (400 oggetti delle vecchie classi + 20 cavalli mai visti).

13. Strategia di Fine-Tuning: Head Adaptation

Congelamento della Feature Backbone

- **Freezing della Backbone:** I pesi dei moduli di estrazione feature (T-Net, Convoluzioni, Max Pooling) vengono **congelati**. Le rappresentazioni geometriche fondamentali rimangono inalterate.
- **Modifica dell'Ultimo Strato:**
  - Il classificatore finale passa da 40 a **41 uscite**.
  - I primi 40 pesi vengono inizializzati dai pesi pre-addestrati; il 41°-esimo neurone viene inizializzato per la classe `horse`.
- **Vantaggio Computazionale:** Addestramento estremamente veloce, eseguibile in pochi secondi anche su CPU.

14. Implementazione del Fine-Tuning

Head Adaptation con PyTorch

```
def fit_head(original, support, replay, replay_y, seed, steps, lr):
    torch.manual_seed(seed)

    # Nuovo classificatore: 40 categorie originali + cavallo.
    head = nn.Linear(256, 41)

    # Copiamo i pesi delle categorie già conosciute.
    with torch.no_grad():
        head.weight[:40].copy_(original.weight)
        head.bias[:40].copy_(original.bias)

    optimizer = torch.optim.Adam(head.parameters(), lr=lr)
    positives = torch.full((len(support),), 40, dtype=torch.long)

    for _ in range(steps):
        optimizer.zero_grad()
        # Metà del peso ai cavalli, metà alle vecchie categorie.
        loss = 0.5 * nn.functional.cross_entropy(head(support), positives)
        loss += 0.5 * nn.functional.cross_entropy(head(replay), replay_y)
        loss.backward()
        optimizer.step()

    return head.eval()
```

15. Risultati Sperimentali: Il Trade-Off

Analisi Quantitativa delle Metriche

- **Con 1-5 Cavalli:** La rete fatica a generalizzare. La Recall sui cavalli è bassa (< 40%).
- **Con 10-25 Cavalli:** Si raggiunge un ottimo equilibrio: Recall sui cavalli attorno all'80-85% e impatto minimo sulle vecchie classi.
- **Con 50 Cavalli:** La Recall sui cavalli sale al **95%**, ma la *Precision* scende e si nota un calo nell'accuracy delle 40 classi originali.

> **Deduzione:** Aumentare i dati della nuova classe senza aumentare i campioni di replay porta la rete a sviluppare un *bias* verso la classe `horse`, scambiando oggetti complessi delle vecchie classi per cavalli.

16. Seconda App: Dashboard dell'Esperimento

Analisi Comparativa dei Risultati (Streamlit)

- **Interfaccia Esperimento (`localhost:8502`):**
  - Grafici interattivi dell'andamento di Recall, Precision e Accuracy al variare del data budget.
  - Ispezione visiva dei cavalli di test classificati correttamente o erratamente.
  - Confronto diretto delle predizioni fornite dai modelli addestrati con 1 vs 50 cavalli.

17. Analisi Critica: Fallimenti e Lezioni Apprese

Cosa Non Ha Funzionato e Come È Stato Risolto

Iniziale Sbilanciamento della Loss:
Nelle prime prove senza bilanciamento dei pesi della Loss, la rete ignorava completamente il singolo cavallo presentato (Recall = 0%) perché le 40 classi dominavano il gradiente. Risolto introducendo la *Weighted Cross-Entropy Loss*.

L'Illusione della Recall Alta:
Misurare solo la Recall per la nuova classe dava l'illusione che 50 cavalli fossero la scelta perfetta. L'analisi della *Precision* ha rivelato il degrado sulle vecchie classi, evidenziando l'importanza di metriche bilanciate.

18. Valutazione dei Limiti del Modello e del Dataset

Domain Shift e Considerazioni di Generalità

- **Domain Shift tra Dataset:** I cavalli provengono da *Objaverse* (modelli mesh moderni), mentre le 40 classi provengono da *ModelNet40* (CAD sintetici puliti). La differenza nella densità e distribuzione dei punti crea un "gap di dominio".
- **Rumore nei Dati Reali:** Le nuvole di punti reali (es. da sensori LiDAR) contengono occlusioni e rumore che non sono presenti nei dati sintetici puliti testati.
- **Limiti di PointNet:** PointNet elabora ogni punto individualmente prima del pooling; non cattura le strutture geometriche locali dettagliate come architetture più recenti (es. PointNet++ o DGCNN).

19. Cosa è Stato Realizzato

Risultati Concreti del Progetto

- **Pipeline Point Cloud:** caricamento dei dati da **Parquet**, campionamento di **1.024 punti**, centratura e normalizzazione della point cloud.
- **Analisi di PointNet:** studio di **T-Net, feature extraction e Global Max Pooling**, fino al vettore globale di **1.024 feature**.
- **Ispezione Interna:** utilizzo dei **Forward Hooks** per individuare quali punti contribuiscono ai massimi delle feature e visualizzarli sulla point cloud.
- **Fine-Tuning:** aggiunta della nuova classe **horse** passando da **40 a 41 classi**, mantenendo il replay di 200 campioni delle classi originali.
- **Valutazione:** confronto delle prestazioni sullo stesso **test set di 420 oggetti** per verificare il riconoscimento della nuova classe e il mantenimento delle classi già apprese.

20. Conclusioni e Sviluppi Futuri

Sviluppi Futuri

- **Testare più quantità di dati:** ripetere il fine-tuning con un numero maggiore di esempi della classe **horse** per studiare come cambia la capacità di apprendimento del modello.
- **Migliorare il Fine-Tuning:** confrontare l'attuale **Head Adaptation** con un fine-tuning parziale della rete, sbloccando progressivamente alcuni livelli della backbone.
- **Analizzare meglio le Feature:** estendere la visualizzazione delle attivazioni per capire quali parti della **point cloud** influenzano maggiormente la classificazione.
- **Ampliare la valutazione:** ripetere gli esperimenti con **più classi e più oggetti** per verificare se i risultati osservati sul cavallo si generalizzano ad altri casi.

Grazie per l'attenzione